Adaptive Candidate Routing for Distributed Occupant Recognition Across Multiple Building IoT-enabled Smart Spaces.

First Author[1,2*], Second Author[2,3†] and Third Author[1,2†]

[1*]Department, Organization, Street, City, 100190, State, Country.
[2]Department, Organization, Street, City, 10587, State, Country.
[3]Department, Organization, Street, City, 610101, State, Country.

*Corresponding author(s). E-mail(s): iauthor@gmail.com;
Contributing authors: iiauthor@gmail.com; iiiauthor@gmail.com;
†These authors contributed equally to this work.

Abstract

A multi-building camera network performing occupant identification and tracking often has to find out which building a seen individual's face belongs to in order to carry out recognition effectively. A centralized gallery of face embeddings facilitates this recognition task effectively, but it keeps sensitive biometric data in one place. The proposed work discusses a routing strategy that avoids the centralized reference repository, yet effectively recognizes and tracks the occupant. Each building publishes one Bloom filter built from locality-sensitive hashing (LSH) codes of its own occupants. A captured face becomes $\boldsymbol{L}$ codes that are checked against all filters, and only the best-scoring buildings verify the face against their own reference embeddings. On 500 CASIA-WebFace identities in 10 buildings, averaged over 20 random building assignments, the default setting identifies 92.37% of queries correctly, against 95.47% for a centralized search with the same vote rule; an adaptive rule that asks extra buildings only when the ranking is unclear reaches 92.44% and contacts 2.54 buildings on average. The mean face and the accept angle come from reference photographs only; $\boldsymbol{K}$ and $\boldsymbol{P}$ were chosen after seeing test accuracy, so alternatives are reported. The real false-positive rate of the filters is 3.54% instead of the configured 1%; LSH code collisions cause the difference. When the same people are regrouped into more and smaller buildings, the share of queries whose true building is in the shortlist falls from 94.8% at 10 buildings to 87.0% at 100. Someone who holds the filters can tell which building a person belongs to (AUC 0.93). Costs are computed, not measured; encryption and unknown visitors are not evaluated.

Keywords: Occupant identification and tracking, Smart Spaces, Locality-sensitive hashing, Bloom filter, Privacy in data analytics

1 Introduction

When a face is seen at one building of a campus or company, the system often has to find out who the person is and where that person is registered. The usual answer is one central gallery with the face embeddings of everyone. This is simple and accurate, but it stores sensitive biometric data in a single place, which is an attractive target and may conflict with data-minimization rules. The other extreme is a gallery in each building and a request to every building for every query, which shows every query to every building.

We study the step between these two: a routing step that decides which few buildings should be asked. Each building publishes one Bloom filter [1] built from LSH codes of its own occupants. A query is checked against all filters, and only the best-scoring buildings receive it. Reference embeddings are not published. The idea of publishing a compact summary of what each site holds, and checking the summaries before contacting a site, is used in web cache sharing [2]. Face matching is approximate, so the summary cannot hold exact keys; it holds LSH codes instead.

Scope. This paper is about the routing step. We measure how accurate it is, how many buildings it contacts, how many false positives the filters give, and how it behaves when there are more buildings. All queries come from enrolled people; identifying unknown visitors is left for future work. We do not implement or test any cryptographic protection. We assume that messages between buildings use a standard authenticated, encrypted channel, and Section 3 states what is and is not protected. Storage and communication costs are computed from message sizes; no distributed run was measured.

Our contributions are:

- a routing design with one Bloom filter of LSH codes per building, and a way to choose the code length K and the number of codes L from measured angle distributions;
- measurements on 500 identities and 20 building assignments: accuracy against a centralized search, the gain from multi-probe hashing, and a real false-positive rate of 3.54% that we trace to collisions between LSH codes;
- a rule that asks extra buildings only when the two best scores are close, which is as accurate as asking all shortlisted buildings and contacts about half as many;
- an evaluation with up to 100 buildings, an analytic cost comparison, and a measurement of what the published filters reveal about who is in a building.

2 Related Work

Existing studies in the domain of indoor occupant tracking focus on camera-based [3]or RF-signal-based approaches [4]. These works highlight computational efficiency

and privacy preservation as two major challenges in occupant identification and tracking across multi-building indoor smart spaces, especially when camera networks are employed [5]. Consequently, we review key studies that address privacy concerns and computational requirements associated with occupant tracking in IoT-enabled multi-building indoor smart spaces.

Occupant identification and tracking are essential to context-aware applications in smart-space environments. However, smart-space environments become more intelligent and context-aware through the integration of IoT-enabled sensing devices [6]. Facial biometrics are particularly well suited for camera-based human identification and tracking in smart spaces because they provide distinctive identity information and are easily captured remotely. Unlike many other biometric modalities, face recognition can operate unobtrusively without requiring the individual to stop, actively cooperate, or explicitly present an identifying credential.

Face identification with secure computation. Erkin et al. [7] let one party with a face image and another party with a database of face templates run Eigenfaces recognition, using homomorphic encryption and secure multiparty computation, so that the database holder learns neither the image nor the result. Sadeghi et al. [8] combine homomorphic encryption with garbled circuits and need fewer rounds, less communication and less computation for the same task. These protocols protect one query against one database holder. We address a different part of the problem: finding which of many sites holds the person.

Biometric template protection and Bloom filters. Jain et al. [9] survey ways to protect stored biometric templates. Rathgeb et al. [10] apply Bloom filters to iris templates. We use Bloom filters differently: one filter summarizes all occupants of a building, and it is used to choose which building to ask.

Approximate nearest-neighbour search. LSH [11, 12], multi-probe LSH [13], product quantization [14] and GPU libraries [15] make search over large galleries fast. They assume that one machine or one organization holds the whole gallery.

Bloom-filter summaries in distributed syables can be inserted via thstems. Summary Cache [2] lets web proxies exchange Bloom filters of their contents and ask only the proxies whose filter matches; Broder and Mitzenmacher [16] survey such uses. Our routing step follows this idea, with LSH codes in place of exact keys.

Camera networks. Mohan and Menon [3] model the identification and tracking of occupants across buildings as a distributed state transition system. We study how a query can be routed to the building that holds the person. To our knowledge, the combination of LSH codes and per-building Bloom filters has not been measured for this task before.

3 System and Threat Model

There are B buildings. Each building holds the reference embeddings of its $N = 50$ occupants ($R = 20$ photographs each) and never publishes them. A query is a photograph of a person who is enrolled in exactly one building. We assume that the buildings follow the protocol but may try to learn from what they see, and that

messages between buildings use an authenticated, encrypted channel. This channel is assumed; we do not implement or evaluate it.

What the design avoids. No building publishes its reference embeddings, there is no gallery in one central place, and no reference embedding is sent between buildings during routing or verification. We do not claim that the published filters are irreversible or that they give biometric privacy.

What it does not protect. A filter holds hashed LSH codes of the occupants, not their embeddings. Still, anyone who holds the filters, the public hash parameters and a photograph of a person can compute the same routing score that routing uses, and can therefore test whether that person belongs to a building; Section 6.6 measures how well. This leakage is the signal that routing needs, so it cannot be removed without losing routing. Guessing candidate faces and using outside information could leak more, and we did not study them. A shortlisted building also receives the query embedding. Malicious or colluding buildings, attacks that guess embeddings from many queries, and traffic analysis are outside the scope of this paper. In a full system, a building that confirms a visitor may keep that person's reference embeddings for the length of the visit; this does share templates, it is outside the scope of this paper, and it is not evaluated.

4 Method

4.1 Embedding and Preprocessing

Faces are detected with SCRFD-10GF [17] and converted by ArcFace [18] (InsightFace buffalo_l, pretrained and not trained by us) into a vector $\tilde{x} \in \mathbb{R}^{512}$, which is scaled to unit length and called x. For unit vectors, $u \cdot v = \cos\theta$, so $\theta = \arccos(u \cdot v)$ is the angle between two faces.

ArcFace embeddings lie in a narrow cone, so hyperplanes through the origin would put almost all of them on the same side. We subtract the population mean $\bar{x}$ and scale to unit length again,

$$\mathrm{pre}(x) = \frac{x - \bar{x}}{\|x - \bar{x}\|}, \tag{1}$$

using the same $\bar{x}$ at every building and for every query.

4.2 Random-Hyperplane LSH

Let H be $K \cdot L$ vectors in $\mathbb{R}^{512}$ drawn independently from a standard Gaussian with a fixed seed that all buildings share. Each vector h gives one bit: $b = 1$ if $x \cdot h \geq 0$, and $b = 0$ otherwise. Groups of K consecutive bits are packed into an integer,

$$c_\ell = \sum_{i=0}^{K-1} b_{\ell,i}\, 2^i \in [0, 2^K), \qquad \ell = 0, \ldots, L-1, \tag{2}$$

which gives L codes per face. So that a match counts only within its own slot, each code is combined with its slot number into an *item*, $s_\ell = \ell \cdot 2^K + c_\ell$. This is the

angular LSH family of Charikar [12]: for unit vectors at angle θ (in degrees), a random hyperplane gives them the same bit with probability

$$p_{\text{bit}} = 1 - \frac{\theta}{180}. \tag{3}$$

We use this family, and not MinHash [19] or bit sampling, because ArcFace embeddings are dense real vectors that are compared by angle.

4.3 Choosing K and L

Two codes match only if all K bits agree, which has probability p_{bit}^K. We average this value over the array of measured pair angles, separately for genuine pairs (same person) and impostor pairs (different people), to get $p_{\text{code}}^{(G)}$ and $p_{\text{code}}^{(I)}$. The average is taken *after* raising to the power K. The function $x \mapsto x^K$ is convex, so pairs with small angles dominate the average; using the mean angle first gives a value of $p_{\text{code}}^{(G)}$ that is too low. The L codes use independent hyperplanes, so

$$\text{recall} = 1 - \left(1 - p_{\text{code}}^{(G)}\right)^L, \qquad \text{FPR}_{\text{imp}} = 1 - \left(1 - p_{\text{code}}^{(I)}\right)^L, \tag{4}$$

and for a recall target of 0.90

$$L = \left\lceil \frac{\ln(1 - 0.90)}{\ln\left(1 - p_{\text{code}}^{(G)}\right)} \right\rceil. \tag{5}$$

For each K from 4 to 15 we solve for L, drop values of K with $\text{FPR}_{\text{imp}} > 0.10$, and keep the smallest $K \cdot L$. The pairs are measured on reference photographs only (Section 5). This model picks $K = 10$, $L = 88$: for $K = 10$ it gives $p_{\text{code}}^{(G)} = 0.0259$ and $\text{FPR}_{\text{imp}} = 0.087$, for $K = 11$ it needs $L = 122$ (0.0187 and 0.062), and for $K = 13$ it needs $L = 230$ (0.0100 and 0.031). The model looks at each pair on its own, but a real filter holds 50 people, so we also run the full pipeline with the model's choice and with $K = 11$, $L = 122$ (Section 6.1).

4.4 Per-Building Bloom Filter

Each building publishes a Bloom filter of its occupants' codes. The 20 preprocessed reference embeddings of an occupant are averaged and scaled to unit length to give one centroid, which is encoded into L items. A building therefore stores $n = N \cdot L = 6100$ items at $L = 122$. The filter size follows the standard formula for a target false-positive rate $f^\star = 0.01$,

$$m = \left\lceil \frac{-n \ln f^\star}{(\ln 2)^2} \right\rceil = 58{,}469, \qquad \kappa = \text{round}\left(\frac{m}{n} \ln 2\right) = 7, \tag{6}$$

which is about 7.3 KB per building. The filter is always sized for $N = 50$ occupants, so its length does not reveal how many people a building really has.

Bit positions are computed with Kirsch–Mitzenmacher double hashing [20]. Two hashes of the item, with salts derived from the shared seed σ, are combined as

$$h_1 = \mathrm{SM64}(s \oplus (2\sigma+1)), \quad h_2 = \mathrm{SM64}(s \oplus (2\sigma+2)) \vee 1, \quad p_j = \big((h_1 + j\, h_2) \bmod 2^{64}\big) \bmod m, \tag{7}$$

for $j = 0, \ldots, \kappa - 1$, where SM64 is the SplitMix64 finalizer [21]. The sum wraps around at 64 bits before the reduction modulo m. Setting the lowest bit of h_2 makes it odd, so it is never zero and the κ positions do not all land on the same bit. ADD sets the κ bits of an item, and CHECK reports an item as present only if all κ bits are set. There are no false negatives. We wrote the Bloom filter and the LSH encoder ourselves in NumPy, without external hashing or Bloom-filter libraries.

4.5 Multi-Probe Routing

With a recall target of 0.90, about one query in ten matches none of its own building's codes exactly, and the true building falls out of the shortlist. When a code misses, it is usually because one bit flipped, and that bit is typically the one whose projection $|x \cdot h|$ was closest to zero. Following multi-probe LSH [13], for each slot we also look up the codes obtained by flipping, one at a time, each of the P bits with the smallest margin,

$$s'_{\ell,j} = \ell \cdot 2^K + \big(c_\ell \text{ XOR } 2^{w_{\ell,j}}\big), \quad j = 1, \ldots, P, \tag{8}$$

where $w_{\ell,j}$ is the index of the j-th weakest bit of slot ℓ. This gives $L(1+P)$ lookups per filter. The routing score of a building is the number of *different slots* with at least one item present, so it stays between 0 and L. The $S = 5$ buildings with the highest scores form the shortlist, and ties are broken by building order. The size of the shortlist is fixed.

4.6 Verification and Policies

A shortlisted building compares the query with its private reference embeddings. A reference *votes* if its angle to the query is at most τ. The occupant with the most votes wins (ties go to the smallest single angle), and the match is accepted only if the winner has at least $V = 12$ of 20 votes. Otherwise the system abstains. The accept angle τ is the cutoff that gives the smallest sum of the genuine false-reject rate and the impostor false-accept rate. The centralized baseline applies the same vote to every enrolled occupant.

We use two ways to choose the final answer among the shortlisted buildings, and the results below always say which one is used. In the *default rule*, the winner is the occupant with the most votes among all occupants of the S shortlisted buildings, and ties go to the smallest angle. This rule gives the multi-probe results and the headline number of the abstract. In the *policies* (Section 4.6), buildings are asked one by one, each asked building returns its own best occupant, and ties between buildings are decided by rank order. *Sequential*: ask the buildings in rank order and stop at the first one whose best vote reaches V; a wrong building ranked above the true one can then confirm first. *Query-all*: ask all S buildings and take the highest vote. *Adaptive*: let

g be the difference between the two highest routing scores. If $g \leq g_0$, the ranking is unclear and all S buildings are asked; otherwise the sequential rule is used.

5 Experimental Setup

We use 500 identities of CASIA-WebFace [22], spread over $B = 10$ buildings of 50 people. Each person has 20 enrolled reference photographs and 20 held-out test photographs, which gives 20,000 embeddings. Every one of the 10,000 test photographs is a query and is routed over all B buildings. The evaluation is *closed-set*: every query belongs to an enrolled person. We use $K = 11$, $L = 122$, $P = 3$, $S = 5$, $V = 12$ and $f^\star = 0.01$.

Use of test photographs. The mean face, the accept angle and the angle statistics behind K and L are estimated from reference photographs only: the mean face is the mean of the 10,000 reference embeddings, genuine pairs are two reference photographs of the same person, and impostor pairs are reference photographs of different people. This gives $\tau = 80.43°$. An earlier version of these experiments used test photographs for the mean face and for τ (80.54°). With the reference-only estimates, correct-identification rates changed by at most 0.06 points, and misidentification rates fell by 0.1 to 0.15 points because τ is lower. The values $K = 11$ and $P = 3$ were chosen after looking at test accuracy, so we report $K = 10$, the choice of the model, next to $K = 11$, and the whole sweep over P (Section 6.1).

Seeds. Each result is repeated for 20 seeds. Seed s changes three things together: the random assignment of the 500 identities to buildings of 50, the LSH hyperplanes, and the Bloom filter salt. The embeddings, the mean face and the accept angle stay the same, so the centralized baseline is a single number. We report the mean over seeds with a 95% confidence half-width ($\pm$, Student-t with 19 degrees of freedom) and, for comparisons, the paired difference over seeds. For $B = 2$ and $B = 5$ we take the identities of the first two and the first five buildings of the original layout and regroup them into buildings of 50. A single run with the original assignment and hyperplane seed 42 gave 92.32% at $P = 3$ with the default rule, 0.05 points below the 20-seed mean; 14 of the 20 seeds reach that value.

Metrics. Every query ends in exactly one of four outcomes: *correct*; *misidentified* (a wrong person is accepted, the most serious error); *abstained-reject* (the true building was shortlisted but nobody reached V votes); and *abstained-routing* (the true building was not shortlisted and no one reached V votes). The centralized baseline gives 95.47% correct, 1.41% misidentified and 3.12% abstained.

6 Results

6.1 Choice of K and L

The analytic model uses only the angle arrays, so it has no seed. Using the mean angle first gives $p_{\text{code}}^{(G)} = 0.0115$ at $K = 11$, against the correct array average of 0.0187, which is about 39% too low. The model picks $K = 10$, $L = 88$. In the full pipeline at $P = 3$ with the default rule, $K = 11$, $L = 122$ is more accurate than $K = 10$ by 0.52 ± 0.13 points (paired) and better in 20 of 20 seeds: correct 92.37% against 91.85%, mainly

because routing failures fall from 4.88% to 4.46%. Both results are reported; we use $K = 11$ in the rest of the paper, and we note that this choice was made after seeing test accuracy.

6.2 Multi-Probe and Decentralized versus Centralized Identification

Table 1 varies P at $B = 10$ with the default rule. Without probing, $8.07 \pm 0.11\%$ of queries are lost to routing misses. Multi-probe with $P = 3$ reduces this to $4.46 \pm 0.08\%$ (a 45% relative reduction), raises correct identification from 88.28% to 92.37% (a paired gain of 4.09 ± 0.15 points, positive in 20 of 20 seeds) and lowers misidentification from 1.91% to 1.42%. The gains get smaller as P grows, while the number of lookups grows linearly, so we use $P = 3$ as a compromise on cost, not as the best possible value.

Table 1 Multi-probe sweep at $B = 10$ ($K = 11$, $L = 122$, default rule). Results are reported as the mean over 20 seeds ($\pm$ 95% CI), in %. The number of items looked up per filter and query is $L(1+P)$.

P	Correct	Misid.	Abst.-reject	Abst.-routing	Items
0	88.28 ± 0.13	1.91 ± 0.04	1.74 ± 0.03	8.07 ± 0.11	122
1	90.92 ± 0.09	1.61 ± 0.04	1.71 ± 0.04	5.76 ± 0.11	244
2	91.90 ± 0.09	1.45 ± 0.04	1.73 ± 0.05	4.92 ± 0.10	366
3	92.37 ± 0.08	1.42 ± 0.03	1.75 ± 0.04	4.46 ± 0.08	488
5	92.80 ± 0.07	1.38 ± 0.03	1.77 ± 0.05	4.05 ± 0.07	732
7	92.89 ± 0.07	1.38 ± 0.03	1.76 ± 0.05	3.97 ± 0.08	976

We now compare the decentralized system with the centralized baseline. At $B = 2$ and $B = 5$ the shortlist of $S = 5$ covers every building, so routing cannot fail and the two systems give the same result in every seed (95.75% correct at $B = 2$ and 95.64% at $B = 5$). At $B = 10$ the decentralized system is 3.10 ± 0.08 points behind the baseline, and 4.46 points of queries are routing failures. Before multi-probe the gap was 7.19 points, so multi-probe closes 57% of it. Misidentification (1.42% against 1.41%) hardly changes, so the loss comes from missed matches and not from more wrong accepts.

6.3 Effective False-Positive Rate

The 1% target describes the bit array for items that are independent of the stored ones. We measured the rate the system really has by building all ten filters and testing the enrolled items of the *other nine* buildings against each one. The real per-item rate is $3.54 \pm 0.02\%$ over the 20 seeds. When we compare the integer codes directly, without a Bloom array, $2.61 \pm 0.02\%$ of the items of other buildings already equal an enrolled item in the same slot. This is close to the 2.41% expected if the codes of 50 occupants were evenly spread over $2^{11} = 2048$ values, namely $1 - (1 - 2^{-K})^N$. The Bloom array alone, tested on random integers that were never inserted, gives $0.94 \pm 0.01\%$ at 51.3% fill, close to its design rate. The extra false positives therefore come from the LSH codes, not from the Bloom array.

A larger K gives more possible codes and lowers the real false-positive rate, but L must grow to keep recall at 0.90. At $K = 13$, $L = 230$ each filter grows from 7.3 KB to about 13.8 KB and the real rate falls to 1.67%. With $P = 6$ it is 0.21 ± 0.08 points more accurate than the best $K = 11$ setting ($P = 7$, 92.89%; paired, better in 19 of 20 seeds) and 0.73 ± 0.11 points more accurate than the default (93.10% against 92.37%; better in 20 of 20 seeds), but it needs 1,610 lookups per filter and query against 488 or 976.

6.4 Verification Policies

Table 2 compares the policies of Section 4.6 on the same 10,000 queries. These results use the policies' own rule for choosing among buildings (best occupant per building, ties by rank order), so query-all gives 92.34% here and not the 92.37% of the default rule in Table 1. The two differ only when two shortlisted buildings return the same number of votes, which happens for 72.5 of 10,000 queries on average. Every query is routed over all ten buildings, including its own, so there is no separate local-recognition stage. Sequential first-match raises the misidentification rate to 2.17%, because a wrong building ranked above the true one sometimes confirms first. Asking all five buildings lowers it to 1.45% and gains 0.71 ± 0.03 points, but it contacts 5 buildings per query. The adaptive rule with $g_0 = 5$ is slightly more accurate than asking all five ($+0.10 \pm 0.02$ points, paired, better in 20 of 20 seeds) while contacting 2.54 buildings on average; 37.6% of the queries ask all five. All three policies abstain on exactly the same queries, because acceptance depends only on whether some shortlisted building reaches V votes.

Table 2 Verification policies at $B = 10$ using the policy rule. Results are reported as the mean over 20 seeds (± 95% CI). Querying every building would contact 10 buildings.

Policy	Correct (%)	Misid. (%)	Buildings asked	Asked all five
Sequential first-match	91.62 ± 0.08	2.17 ± 0.04	1.50	–
Adaptive, $g_0 = 5$	$\mathbf{92.44 \pm 0.09}$	1.35 ± 0.04	2.54	37.6%
Query all five	92.34 ± 0.09	1.45 ± 0.04	5.00	100%

We repeated the comparison at two and five buildings (the same people regrouped into buildings of 50 for each seed; the shortlist is every building). Table 3 shows that the adaptive rule helps wherever ranking errors occur. At $B = 2$ the gain over sequential first-match is small (0.07 ± 0.03 points, better in 14 of 20 seeds), because with two buildings a wrong building rarely outranks the true one. At $B = 5$ the adaptive rule gains 0.64 ± 0.05 points over sequential first-match (better in 20 of 20 seeds), is $+0.01 \pm 0.02$ points above asking all five buildings, and contacts 2.29 rather than 5 buildings on average (54% fewer). Misidentifications fall from 1.37% (sequential) to 0.74% (adaptive). Query-all is within 0.01 points of the centralized search at $B = 5$ (95.63% against 95.64%).

Table 3 Verification policies at $B = 2$ and $B = 5$ using the policy rule. Results are reported as the mean over 20 seeds (± 95% CI). Query-all asks every building.

	$B = 2$			$B = 5$		
Policy	Correct (%)	Asked	All	Correct (%)	Asked	All
Sequential first-match	95.66 ± 0.04	1.09	–	95.01 ± 0.06	1.35	–
Adaptive, $g_0 = 5$	95.73 ± 0.02	1.20	18.0%	$\mathbf{95.64 \pm 0.02}$	2.29	31.4%
Query all	95.75 ± 0.01	2.00	100.0%	95.63 ± 0.01	5.00	100.0%

6.5 Cost

The cost figures are computed from message sizes and from the measured number of buildings asked; they are not measurements. We did not measure latency, throughput or traffic in a distributed run, because all calls were in-process. A query embedding has 512 single-precision values, which is 2,048 bytes. Routing replaces a 20.5 MB gallery in one place with ten published filters of 7.3 KB each (the buildings keep 2.05 MB of private embeddings each). A query then costs 3.1 KB with sequential asking (1.50 buildings), 5.2 KB with the adaptive rule (2.54 buildings) and 10.2 KB when all five shortlisted buildings are asked, against 20.5 KB when all ten buildings are asked and 2.0 KB for a central gallery. Each query still costs $B \cdot L(1 + P) = 4{,}880$ filter lookups, a cost that grows linearly with B.

6.6 What the Filters Reveal

We measured the simplest attack. Someone who holds all published filters and the public hash parameters scores a photograph against every filter, in the same way as routing does. With $P = 3$, this score separates the occupants of a building from everyone else with an AUC of 0.93 (0.88 without probing), averaged over buildings and 10 seeds, and the highest-scoring building is the person's own building for 79.7% of the photographs (66.7% without probing). The filters therefore leak which building a person belongs to. This is not a way to recover a face, but it is a real leak, and it grows with the accuracy of the routing itself. We did not test attacks that use many queries, candidate guessing or outside information, so we make no claim of biometric privacy.

6.7 More Buildings

The corpus has only 500 identities, so a larger network can only be built with smaller buildings. We regroup the same 500 people into $B = 10$, 25, 50 and 100 buildings of 50, 20, 10 and 5 people. Each filter is sized for its own number of occupants, so the total size of all filters stays the same (about 73 KB). We use 2,000 queries per seed (4 photographs per person) and 10 seeds. Table 4 shows that the share of queries whose true building is in the shortlist falls from 94.8% to 87.0%, and correct identification falls from 92.5% to 86.7% (adaptive policy), while the centralized search stays at 95.5%. The adaptive policy still asks about 2.5 buildings. The real false-positive rate of a filter falls with building size, from 3.54% to 1.27%, so the loss comes from having to pick 5 buildings out of more of them, not from more false hits per filter. Lookups

per query grow in proportion to B (from 4,880 to 48,800). These results are for smaller buildings and a fixed set of people; they do not replace a test with a larger population.

Table 4 Growth in the number of buildings (same 500 people, $K = 11$, $L = 122$, $P = 3$, $S = 5$): mean over 10 seeds, in %. Recall is the share of queries whose true building is in the shortlist.

B	people/bldg	recall	correct (adaptive)	correct (all five)	real FPR	filter size
10	50	94.8 ± 0.3	92.5 ± 0.3	92.4 ± 0.3	3.54%	7.3 KB
25	20	90.7 ± 0.4	89.8 ± 0.5	89.7 ± 0.5	2.04%	2.9 KB
50	10	89.1 ± 0.2	88.6 ± 0.3	88.6 ± 0.3	1.52%	1.5 KB
100	5	87.0 ± 0.4	86.7 ± 0.4	86.7 ± 0.4	1.27%	0.7 KB

7 Limitations and Conclusion

Limitations. All results use the same 500 identities and photographs from one public dataset; the intervals describe the effect of building assignments and hash randomness, not a different population. The values $K = 11$ and $P = 3$ were chosen after seeing test accuracy, and we report the alternatives next to them. CASIA-WebFace photographs come from the web and not from surveillance cameras, so the absolute accuracy is probably higher than it would be in practice. The ArcFace model was pretrained on a large public face collection, and we did not check how much it overlaps with CASIA-WebFace. The largest network has 100 small buildings. The evaluation is closed-set, so we did not measure what happens when a person who is enrolled nowhere, such as a visitor, is seen. The vote rule (τ and $V = 12$) was chosen for enrolled people, and a first test with held-out people showed that it can accept a noticeable share of them, so the acceptance rule has to be studied and tightened before the system is used with unknown visitors. Storage and communication costs are computed from message sizes; we did not measure latency or traffic in a distributed run. We did not implement or evaluate the encrypted channel, the visit-length storage of templates or attacks on the filters beyond the membership test of Section 6.6. A Bloom filter cannot delete items, so a change in enrollment needs a rebuild.

Conclusion. Per-building Bloom filters of LSH codes can route face queries to the right building without a central gallery. With multi-probe hashing the default setting identifies 92.37% of queries correctly at 10 buildings against 95.47% for a centralized search, and the adaptive rule reaches 92.44% while contacting about 2.54 buildings. The cost is lost recall that grows with the number of buildings, and a false-positive rate that LSH collisions raise well above the Bloom filter's design value. The privacy conclusions are limited to what was evaluated: the design avoids publishing reference embeddings and a central gallery, and it leaks which building a person belongs to (AUC 0.93); encryption, stronger attacks and template storage during visits were not evaluated. The next steps are a larger and more varied population, the identification of unknown visitors with a stricter acceptance rule, and measurements in a distributed deployment.

References

[1] Bloom, B.H.: Space/time trade-offs in hash coding with allowable errors. Communications of the ACM **13**(7), 422–426 (1970)

[2] Fan, L., Cao, P., Almeida, J., Broder, A.Z.: Summary cache: A scalable wide-area web cache sharing protocol. IEEE/ACM Transactions on Networking **8**(3), 281–293 (2000)

[3] Mohan, L., Menon, V.: Modelling large scale camera networks for identification and tracking: An abstract framework. IET Computer Vision **14**(7), 426–433 (2020)

[4] Wu, Z., Zhang, D., Shang, Z., Yuan, Y., Gong, H., Wang, B., Lu, Z., Li, Y., Hu, Y., Sun, Q., Chen, Y.: Learning-based tracking-before-detect for unconstrained indoor human tracking using rf signal. IEEE Transactions on Mobile Computing **24**(6), 4777–4793 (2025) https://doi.org/10.1109/TMC.2025.3529501

[5] Kim, J., Kim, Y.M., Zahreddine, R., Welge, W.A., Krishnan, G., Ma, S., Wang, J.: Privacy-preserving visual localization with event cameras. IEEE Transactions on Image Processing **34**, 6215–6230 (2025) https://doi.org/10.1109/TIP.2025.3607640

[6] Khan, I., Zedadra, O., Guerrieri, A., Spezzano, G.: Occupancy prediction in iot-enabled smart buildings: Technologies, methods, and future directions. Sensors **24**(11), 3276 (2024)

[7] Erkin, Z., Franz, M., Guajardo, J., Katzenbeisser, S., Lagendijk, I., Toft, T.: Privacy-preserving face recognition. In: Goldberg, I., Atallah, M.J. (eds.) Privacy Enhancing Technologies. Lecture Notes in Computer Science, vol. 5672, pp. 235–253. Springer, Heidelberg (2009)

[8] Sadeghi, A.-R., Schneider, T., Wehrenberg, I.: Efficient privacy-preserving face recognition. In: Lee, D., Hong, S. (eds.) Information Security and Cryptology – ICISC 2009. Lecture Notes in Computer Science, vol. 5984, pp. 229–244. Springer, Heidelberg (2010)

[9] Jain, A.K., Nandakumar, K., Nagar, A.: Biometric template security. EURASIP Journal on Advances in Signal Processing **2008**, 579416 (2008)

[10] Rathgeb, C., Breitinger, F., Busch, C., Baier, H.: On application of bloom filters to iris biometrics. IET Biometrics **3**(4), 207–218 (2014)

[11] Indyk, P., Motwani, R.: Approximate nearest neighbors: Towards removing the curse of dimensionality. In: Proceedings of the 30th Annual ACM Symposium on Theory of Computing, pp. 604–613. ACM, ??? (1998)

[12] Charikar, M.S.: Similarity estimation techniques from rounding algorithms. In: Proceedings of the 34th Annual ACM Symposium on Theory of Computing, pp. 380–388. ACM, ??? (2002)

[13] Lv, Q., Josephson, W., Wang, Z., Charikar, M., Li, K.: Multi-probe LSH: Efficient indexing for high-dimensional similarity search. In: Proceedings of the 33rd International Conference on Very Large Data Bases, pp. 950–961. VLDB Endowment, ??? (2007)

[14] Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. IEEE Transactions on Pattern Analysis and Machine Intelligence **33**(1), 117–128 (2011)

[15] Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. IEEE Transactions on Big Data **7**(3), 535–547 (2021)

[16] Broder, A., Mitzenmacher, M.: Network applications of bloom filters: A survey. Internet Mathematics **1**(4), 485–509 (2004)

[17] Guo, J., Deng, J., Lattas, A., Zafeiriou, S.: Sample and computation redistribution for efficient face detection. In: International Conference on Learning Representations (2022)

[18] Deng, J., Guo, J., Xue, N., Zafeiriou, S.: ArcFace: Additive angular margin loss for deep face recognition. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4690–4699. IEEE, ??? (2019)

[19] Broder, A.Z.: On the resemblance and containment of documents. In: Proceedings of Compression and Complexity of Sequences 1997, pp. 21–29. IEEE, ??? (1997)

[20] Kirsch, A., Mitzenmacher, M.: Less hashing, same performance: Building a better bloom filter. In: Azar, Y., Erlebach, T. (eds.) Algorithms – ESA 2006. Lecture Notes in Computer Science, vol. 4168, pp. 456–467. Springer, Heidelberg (2006)

[21] Steele, G.L., Lea, D., Flood, C.H.: Fast splittable pseudorandom number generators. In: Proceedings of OOPSLA 2014, pp. 453–472. ACM, ??? (2014)

[22] Yi, D., Lei, Z., Liao, S., Li, S.Z.: Learning Face Representation from Scratch (2014)